

Hands-On 9

COMPILAZIONE MODULARE, SYMBOL TABLE, MAKE

Sandi Russo

Corso di Laurea: Scienze Informatiche

26 giugno 2026

Indice

1	Introduzione	3
2	Definizione del Problema	3
3	Metodologia	3
3.1	Parte 1: Analisi Singolo File	3
3.2	Parte 2: Analisi Programma Modulare	5
4	Presentazione dei Risultati	6
4.1	Risultati Parte 1: Singolo File	6
4.1.1	Analisi Output <code>nm main.o</code>	6
4.1.2	Analisi Output <code>size main.o</code>	7
4.2	Risultati Parte 2: Programma Modulare	7
4.2.1	Codici Sorgente Moduli	7
4.2.2	Makefile Commentato	7
4.2.3	Analisi Output <code>make check</code>	8
5	Conclusione	8

1 Introduzione

L'obiettivo principale di questo report è familiarizzare con i concetti fondamentali della **programmazione modulare** in C, analizzare il processo di **compilazione separata** (senza linking), comprendere la struttura e il significato della **tabella dei simboli** (*Symbol Table*) nei file oggetto, e utilizzare l'utility make per l'**automazione della compilazione** di progetti composti da più file sorgente. Le osservazioni raccolte sono presentate come richiesto dalle specifiche dell'esercitazione.

2 Definizione del Problema

Questa esercitazione si articola in due parti principali, ciascuna mirata a esplorare aspetti specifici della compilazione e gestione di codice C:

1. Analisi di un Singolo File Sorgente:

- **Compito:** Compilare un file C fornito (`main.c`) *senza* eseguire la fase di linking, generando un file oggetto (`.o`).
- **Obiettivo:** Osservare e analizzare la tabella dei simboli del file oggetto risultante tramite il comando `nm`. È richiesta la comprensione e giustificazione di ogni colonna dell'output di `nm` (indirizzo, tipo del simbolo - b, d, t, u, r, ecc. - e nome del simbolo), facendo riferimento anche alla documentazione (`man nm`).
- **Estensione:** Modificare il codice sorgente aggiungendo o eliminando variabili casualmente e utilizzare il comando `size --format=GNU` sul file oggetto per osservare e spiegare la variazione delle dimensioni dei segmenti `text`, `data` e `bss`.

2. Analisi di un Programma Modulare con make:

- **Compito:** Utilizzare l'utility make e un Makefile fornito per compilare un programma C diviso in più moduli (`header.h`, `file1.c`, `file2.c`, `main.c`).
- **Obiettivo:** Osservare le tabelle dei simboli dei singoli file oggetto generati (`file1.o`, `file2.o`, `main.o`) tramite il target make `check` (che invoca `nm`). Comprendere e giustificare l'output di `nm` per ciascun file oggetto, con particolare attenzione ai simboli definiti, non definiti (Undefined), locali e globali, e a come la modularità influisce sulla tabella dei simboli prima del linking.

Il report deve documentare i comandi utilizzati, i codici sorgente, l'analisi degli output e le conclusioni tratte.

3 Metodologia

Per affrontare i problemi definiti, sono stati seguiti i seguenti passaggi metodologici:

3.1 Parte 1: Analisi Singolo File

1. **Codice Sorgente:** È stato utilizzato il codice C fornito nell'esercitazione (Codice 1).

```

1      #include <stdio.h>
2
3      int a;
4      static int b = 10;
5
6      void funzione() {
7          static int c = 5;
8          int d = 20;
9          c++;
10         d++;
11     }
12
13     int main() {
14         funzione();
15         return 0;
16     }
17

```

Listing 1: Codice C per l'esercizio 1 (main.c)

2. **Compilazione Separata:** Il codice è stato compilato senza linking usando GCC con i flag `-e -s -c`.

```
gcc -e -s -c main.c
```

- **-e: preprocessor:** questo parametro fa in modo che il compilatore si fermi dopo la fase di elaborazione
- **-s: compiler:** questo parametro fa sì che si generi un file *assembly* (.s) invece del codice oggetto.
Il file contiene le istruzioni assembly corrispondenti al codice C
- **-c: compilazione (senza linking):** traduce il codice sorgente *main.c* in codice oggetto *main.o*.
Il file contiene il codice oggetto, pronto ad essere linkato con altre unità di compilazione.

3. **Analisi Tabella dei Simboli:** La tabella dei simboli del file *main.o* è stata ispezionata con `nm`.

```
nm main.o
```

- a) **indirizzo** (esadecimale): locazione di memoria *relativa* all'interno del segmento corrispondente nel file *.o*. La differenza di indirizzi tra simboli consecutivi dello stesso tipo dipende dalla dimensione del simbolo precedente.

Tabella dei Simboli		
Indirizzo	Tipo	Nome
000000000000000004	C	a
000000000000000000	d	b
000000000000000004	d	c.2317
000000000000000000	T	funzione
000000000000000025	T	main

Tabella 1: Tabella dei Simboli del codice 1

b) **tipo** (simbolo):

Simbolo	Descrizione
T	Simbolo nel segmento <i>Text</i> (codice), solitamente una funzione definita nel file, come il <i>main</i> .
t	Simbolo <i>locale</i> nel segmento <i>Text</i> (funzione statica).
D	Simbolo nel segmento <i>Data</i> (<i>static int b = 10</i>), lavoriamo su un dato inizializzato.
d	Simbolo <i>locale</i> nel segmento <i>Data</i> (<i>static int c = 5</i>), inizializzata e all'interno della funzione.
B	Simbolo nel segmento <i>BSS (Block Started by Symbol)</i> (dati non inizializzati), es: <i>int a</i> , variabile di tipo globale.
b	Simbolo <i>locale</i> nel segmento <i>BSS</i> , <i>static int d</i> , all'interno di una funzione.
U	Simbolo undefined (non definito in questo file, richiesto dal linker), come il <i>printf</i> , se la libreria è su un altro file.
C	Simbolo <i>Common</i> , simile a <i>BSS</i> , per variabili globali non inizializzate.

4. **Analisi Segmenti:** Le dimensioni dei segmenti sono state analizzate con `size`.

```
size --format=GNU main.o
```

5. **Modifiche e Osservazioni:** Sono state introdotte modifiche (aggiunta/rimozione di variabili globali/statiche, inizializzate/non) nel codice, ripetendo i passi 2 e 4 per osservare l'impatto sui segmenti `text`, `data`, `bss`.

3.2 Parte 2: Analisi Programma Modulare

1. **Codici Sorgente:** Sono stati utilizzati i file `header.h`, `file1.c`, `file2.c`, `main.c` e `Makefile` forniti (riportati nella sezione Risultati).
2. **Compilazione con make:** L'intero progetto è stato compilato e linkato usando il comando `make` nella directory contenente i file.

```
make
```

Questo comando legge il `Makefile` e genera i file oggetto (`.o`) necessari e l'eseguibile finale (programma).

3. **Analisi Simboli Moduli:** Le tabelle dei simboli dei singoli file oggetto sono state ispezionate usando il target check definito nel Makefile.

```
make check
```

Questo comando esegue nm su file1.o, file2.o, e main.o.

4. **Pulizia:** I file generati (.o e l'eseguibile) sono stati rimossi usando il target clean.

```
make clean
```

5. **Analisi Makefile:** Il contenuto e la logica del Makefile sono stati analizzati per comprenderne il funzionamento (variabili, regole, dipendenze, comandi).

4 Presentazione dei Risultati

Questa sezione presenta i codici sorgente completi (dove non già mostrati), il Makefile commentato e l'analisi dettagliata degli output ottenuti.

4.1 Risultati Parte 1: Singolo File

4.1.1 Analisi Output nm main.o

L'analisi dell'output di nm main.o (basata sulla metodologia descritta) rivela quanto segue per i simboli principali del codice main.c:

- a: Essendo globale non inizializzata, appare come simbolo C (Common) o B (BSS), indicando che richiederà spazio nel segmento BSS dell'eseguibile finale. L'indirizzo nel file .o è tipicamente zero.
- b: Globale statica inizializzata. Il tipo è d (Data locale), poiché static ne limita la visibilità al file. Occupa spazio nel segmento Data del file .o con un indirizzo relativo.
- funzione: Funzione globale. Il tipo è T (Text globale), rappresenta codice eseguibile e ha un indirizzo nel segmento Text.
- c: Locale statica inizializzata. Il tipo è d (Data locale), occupa spazio nel segmento Data e il suo nome potrebbe avere un suffisso (es. c.1234) per l'univocità interna.
- d: **Non presente** nella tabella dei simboli. Essendo una variabile automatica, viene allocata sullo stack a runtime e non fa parte dei segmenti statici del file oggetto.
- main: Funzione globale. Il tipo è T (Text globale), con un indirizzo nel segmento Text.

L'analisi degli indirizzi conferma che sono offset relativi all'interno dei rispettivi segmenti nel file oggetto.

4.1.2 Analisi Output size main.o

L'output di `size --format=GNU main.o` mostra le dimensioni dei segmenti:

- `text`: Dimensione del codice compilato per funzione e `main`.
- `data`: Dimensione totale delle variabili statiche/globali inizializzate (b e c).
- `bss`: Dimensione dello spazio richiesto per le variabili statiche/globali non inizializzate (a).

L'esperimento di aggiunta/rimozione variabili ha confermato che: variabili inizializzate incidono su `data`, quelle non inizializzate su `bss`, e le funzioni su `text`, come atteso.

4.2 Risultati Parte 2: Programma Modulare

4.2.1 Codici Sorgente Moduli

I codici `header.h`, `file1.c`, `file2.c`, `main.c` sono quelli presentati nella sezione Metodologia o nelle specifiche dell'Hands-On.

4.2.2 Makefile Commentato

Il Makefile utilizzato è cruciale per la gestione del progetto modulare.

```
1 # Variabili
2 CC = gcc                # Compilatore
3 CFLAGS = -g -Wall       # Flag: debug info, tutti i warning
4 TARGET = programma      # Nome eseguibile finale
5 OBJS = main.o file1.o file2.o # File oggetto necessari
6
7 # Regola Default: 'all' dipende dall'eseguibile finale
8 all: $(TARGET)
9
10 # Regola di Linking: Crea l'eseguibile ($(TARGET)) dagli oggetti ($(OBJS))
11 $(TARGET): $(OBJS)
12     $(CC) $(CFLAGS) -o $@ $^ # Linka oggetti ($^) per creare target ($@)
13
14 # Regola Pattern: Crea un file '.o' da un file '.c'
15 %.o: %.c
16     $(CC) $(CFLAGS) -c $< -o $@ # Compila sorgente ($<) per creare oggetto ($@)
17
18 # Regola 'check': Mostra simboli dei file oggetto
19 check: $(OBJS) # Assicuriamoci che gli oggetti esistano prima di check
20     @echo "\n=== Simboli file1.o ==="
21     nm file1.o
22     @echo "\n=== Simboli file2.o ==="
23     nm file2.o
24     @echo "\n=== Simboli main.o ==="
25     nm main.o
26
27 # Regola 'clean': Rimuove i file generati
28 clean:
29     rm -f $(OBJS) $(TARGET) # Forza rimozione oggetti ed eseguibile
30
31
```

```

32 # Target Fittizi: 'all', 'clean', 'check' sono azioni, non file
33 .PHONY: all clean check

```

Listing 2: Makefile commentato (HO9)

Funzionamento:

- make invoca ‘all’, che dipende da ‘programma’.
- Per creare ‘programma’, ‘make’ vede che dipende da ‘main.o’, ‘file1.o’, ‘file2.o’.
- Se un ‘.o’ manca o il ‘.c’ corrispondente è più recente, usa la regola pattern ‘
- Una volta creati tutti gli ‘.o’, usa la regola ‘(*TARGET*) :(*OBJS*)’ per linkarli (‘gcc -g -Wall -o programma main.o file1.o file2.o’).
- make check esegue ‘nm’ sugli oggetti.
- make clean rimuove i file generati.

4.2.3 Analisi Output make check

L’output di ‘make check’ mostra le tabelle dei simboli individuali:

- **nm file1.o:** Mostra `var_extern` come definita (D), `var_static` come locale (d), `modifica_var_ext` come codice (T), e `printf` come indefinita (U).
- **nm file2.o:** Mostra `stampa_var_extern` come codice (T), ma `var_extern` come indefinita (U) perché definita altrove, e `printf` come indefinita (U).
- **nm main.o:** Mostra `main` come codice (T), ma sia `modifica_var_extern` che `stampa_var_extern` come indefinite (U) perché definite in altri moduli.

Questa analisi evidenzia come ogni modulo conosca solo i propri simboli definiti e richieda al linker di risolvere i riferimenti esterni (U) trovando le definizioni negli altri moduli o librerie.

5 Conclusione

Quest’esercitazione ha permesso di consolidare la comprensione di aspetti chiave del processo di compilazione e linking in C, specialmente in contesti modulari.

- L’uso del flag `gcc -c` è fondamentale per la **compilazione separata**, che genera file oggetto contenenti codice rilocabile e metadati come la tabella dei simboli.
- L’utility `nm` è uno strumento prezioso per ispezionare i file oggetto, rivelando quali simboli sono definiti, quali sono referenziati ma non definiti (U), e la loro classificazione (Text, Data, BSS) e visibilità (locale/globale).
- L’utility `size` permette di quantificare l’impatto di diverse scelte di programmazione (variabili inizializzate/non, funzioni) sulla dimensione dei segmenti di memoria text, data, e bss.
- La distinzione tra variabili **automatiche** (stack, non in nm), **statiche locali/globali** (visibilità limitata, segmenti Data/BSS), e **globali** (visibilità estesa, segmenti Data/BSS/Text per funzioni) è cruciale.

- L'utility make e la scrittura di Makefile sono essenziali per **automatizzare la compilazione** di progetti non banali, gestendo le dipendenze tra file e riducendo la possibilità di errori umani nel processo di build.

Queste competenze sono propedeutiche allo sviluppo di progetti software più complessi e strutturati.